Architectural Strategies for LLM Token Optimization:
Maximizing Output Quality While Minimizing Computational Cost

Yuvaraj Akkole
AI/ML Systems Engineering, Cost-Optimized AI Architecture
June 2026

Keywords: LLM optimization, token efficiency, prompt caching, model routing, context engineering, cost reduction, AI architecture, speculative decoding, multi-agent systems, fine-tuning, distillation

Abstract—Large Language Models (LLMs) have become foundational infrastructure for modern software systems, yet their operational costs scale linearly with token consumption—creating an economic tension between capability and sustainability. This paper presents a systematic taxonomy of optimization strategies that achieve **up to 98% cost reduction** while maintaining or improving output quality. We examine techniques spanning prompt engineering, intelligent caching, model routing, context management, reasoning effort control, multi-agent architectures, fine-tuning, and inference infrastructure—synthesizing findings from academic research (LLMLingua, FrugalGPT, RouteLLM), production systems (GitHub Copilot, Claude Code, Cursor), and enterprise deployments. Comprehensive per-provider pricing benchmarks (Anthropic, OpenAI, Google, DeepSeek) and quantified savings tables are provided. Our analysis demonstrates that token optimization is not about using LLMs less, but about using them *architecturally better*—treating token budget as a first-class engineering constraint. The compound application of even a subset of these strategies routinely yields 85–95% cost reductions in production.

1. Introduction

1.1 The Token Economics Problem

The AI industry faces a structural paradox: organisations want to increase LLM integration while controlling exponentially growing inference costs. With frontier models priced at $5–$30 per million input tokens and $15–$180 per million output tokens, a production application processing 100M tokens daily faces costs of $500–$18,000 *per day*. At scale, these figures dominate engineering budgets. Yet the solution is not to reduce LLM usage—the solution is to **engineer systems that extract maximum value per token spent**.

1.2 The Cost Landscape in 2026

The 100–1000× cost spread between model tiers is the single most powerful optimization lever: routing 70% of requests to budget-tier models immediately reduces costs by 70–90%. Key ratios to keep in mind:

- **Frontier vs. Budget:** Claude Opus ($5/$25) is 50× more expensive on input than Gemini 2.5 Flash-Lite ($0.10/$0.40).
- **Cached vs. Uncached:** All major providers offer 90% savings on cached input tokens.
- **Batch vs. Real-time:** Universal 50% discount for async batch processing across all providers.
- **DeepSeek cache hit:** $0.0028/MTok—effectively free for repeated queries.

1.3 Contributions

This paper provides: (1) a hierarchical taxonomy of optimization techniques ordered by impact; (2) quantified benchmarks from peer-reviewed research and production deployments; (3) comprehensive pricing tables across all major providers; (4) architectural patterns for token-efficient AI applications; (5) coding-assistant strategies for GitHub Copilot, Claude Code, and Cursor; and (6) a decision framework for strategy selection.

2. Per-Token Pricing Benchmarks

Tables 1–4 summarise current pricing across all major providers as of mid-2026.

Table 1: Anthropic Claude Pricing ($/MTok)

Model	In	Out	Cache	B.In	B.Out
Opus 4.8	5.00	25.00	0.50	2.50	12.50
Opus 4.7	5.00	25.00	0.50	2.50	12.50
Sonnet 4.6	3.00	15.00	0.30	1.50	7.50
Haiku 4.5	1.00	5.00	0.10	0.50	2.50

Cache: 1.25× base (5-min TTL) write.

Table 2: OpenAI Pricing ($/MTok)

Model	Input	Cached	Output
GPT-5.5	5.00	0.50	30.00
GPT-5.5 Pro	30.00	—	180.00
GPT-5.4	2.50	0.25	15.00
GPT-5.4-mini	0.75	0.075	4.50
GPT-5.4-nano	0.20	0.02	1.25

Batch API: 50% discount on all tiers.

Table 3: Google Gemini Pricing ($/MTok)

Model	Input	Output	Cached
3.1 Pro Preview	2.00	12.00	0.20
3.5 Flash	1.50	9.00	0.15
2.5 Pro	1.25	10.00	0.13
2.5 Flash	0.30	2.50	0.03
2.5 Flash-Lite	0.10	0.40	0.01
2.0 Flash	0.15	0.60	—

Batch/Flex: 50% discount.

Table 4: DeepSeek & Open-Source ($/MTok)

Provider	Model	In	Cache	Out
DeepSeek	V4 Flash	0.140	0.0028	0.280
DeepSeek	V4 Pro	0.435	0.0036	0.870
Together	Llama 4 Scout	0.110	—	0.340
Together	Gemma 3 27B	0.070	—	0.070

3. Prompt Engineering for Token Efficiency

3.1 Prompt Compression

Academic research treats prompt compression as an information-theoretic problem: which tokens carry the most signal for the downstream task?

LLMLingua [1] (EMNLP 2023) introduced a coarse-to-fine pipeline achieving up to **20× compression** with minimal performance degradation across GSM8K, BBH, ShareGPT, and Arxiv datasets.

LLMLingua-2 [2] (2024) achieved 3×–6× faster compression and 1.6×–2.9× end-to-end latency acceleration at 2×–5× compression ratios using XLM-RoBERTa-large.

LongLLMLingua [3] (2024) targets long-context scenarios: **21.4% performance boost with 4× fewer tokens** on NaturalQuestions (GPT-3.5-Turbo) and **94% cost reduction** on the LooGLE benchmark.

3.2 Structural Prompt Design

Four architectural principles for token-efficient prompts:

P1 — Front-load static content. Place system instructions, tool definitions, and reference materials at the beginning of prompts to enable prefix-based caching (Section 4).

P2 — Eliminate format negotiation. Use structured output schemas or pre-fill techniques instead of verbose prose instructions. Claude's pre-fill feature injects the opening of a response (e.g., `{"analysis":`), eliminating preamble tokens entirely.

P3 — Decompose over monolith. AlphaCodium [13] demonstrated that breaking a single complex prompt into a multi-step workflow increased GPT-4 accuracy from **19% to 44%** on code generation while individual steps used fewer tokens.

P4 — Chisel, don't append. Remove every word that does not change the output. Replace verbose instructions with examples; a 3-line example often replaces a 50-word explanation.

3.3 Few-Shot vs. Zero-Shot Economics

A fine-tuned GPT-3.5 (BLEU: 78) outperformed few-shot GPT-4 (BLEU: 70) on Icelandic text correction [8]. For high-volume workloads, fine-tuning eliminates per-request overhead while achieving superior quality. As a rule: if a smaller model with 10-shot prompting outperforms a zero-shot larger model, use it— at 5× lower cost.

4. Intelligent Caching Architectures

4.1 Provider-Level Prompt Caching

Anthropic: Cache reads cost 10% of base input price (90% savings). Cache writes cost 1.25× (5-min TTL) or 2× (1-hour TTL). Supports 4 explicit cache breakpoints per request with a 20-block automatic lookback window; minimum cacheable: 1,024 tokens. Production results: 100K-token book analysis achieved **79% latency reduction, 90% cost savings**; multi-turn conversation yielded **75% faster, 53% cost reduction**.

OpenAI: Fully automatic—no code changes required. Activates for prompts with 1,024+ tokens using prefix-hash matching. Delivers up to **80% latency reduction and 90% input cost savings**.

Optimal request structure (ordered for cache efficiency):

```
[System Instructions  ] <- Static all requests
[Tool Definitions     ] <- Static per session
[Reference Documents  ] <- Static per task type
[Conversation History ] <- Grows, prefix matches
[Current User Message ] <- Unique per request
```

Cache pre-warming: For latency-critical applications, send requests with `max_tokens: 0` before user interaction begins to establish warm cache entries.

4.2 Application-Level Semantic Caching

Semantic caching converts requests to vector embeddings and checks cosine similarity against cached responses (threshold 0.85–0.95). Cost reduction: 50–90% depending on hit rates; speed improvement: 2–4× typical (up to 50–100×). Vector search adds only 5–20 ms overhead while saving 1–5 seconds per query.

Caution: A semantic cache may return the answer for "Mission Impossible 2" when asked about "Mission Impossible 3" — the embeddings are nearly identical but the answers differ completely. Start with deterministic caching (entity-ID, constrained input) before layering semantic matching.

4.3 KV-Cache Optimisation

The **Medusa** framework adds parallel decoding heads sharing the KV-cache, achieving **2.2–3.6× speedup** [6] with no quality loss. **Speculative decoding** further yields a 2–3× throughput improvement at identical quality (see Section 7).

5. Model Routing and Cascading

5.1 FrugalGPT: Cascade Architecture

FrugalGPT [4] (Stanford, 2023) demonstrated that learned LLM cascades can achieve: (1) up to **98% cost reduction** while matching GPT-4 performance, and (2) **4% accuracy improvement** over GPT-4 alone at equivalent cost budgets.

5.2 RouteLLM: Production-Ready Routing

RouteLLM [5] (LMSYS, 2024) routes queries between strong and weak models based on estimated complexity. MT Bench: **85%+ cost reduction** maintaining 95% of GPT-4 quality. MMLU: **45% cost reduction** with equivalent accuracy. Achieves 95% of GPT-4 performance using only **26% GPT-4 calls**— 74% routed to cheaper models.

5.3 Reasoning Effort Control

Modern models expose explicit reasoning budget controls delivering dramatic savings. Claude's `effort` parameter spans `low` through `max`. A complex request at

`max` effort on Claude Opus may consume 30,000+ thinking tokens ($0.75 in thinking alone), versus 0–2,000 tokens at `low` effort ($0.05)— a **15× cost reduction from a single parameter change**.

Table 5: Claude `effort` Parameter Reference

Level	Behaviour	Use Case
`max`	No token constraints	Deep analysis
`xhigh`	Long-horizon work	>30 min tasks
`high`	Default	Complex reasoning
`medium`	Moderate savings	Balanced tasks
`low`	Maximum efficiency	Classification

5.4 Practical Routing Heuristics

Routing signals that work in production:

- **Token count:** Short queries (<50 tokens) → small model.
- **Keyword signals:** "analyse", "compare", "synthesise" → large model.
- **Confidence cascading:** Start small; escalate if confidence < threshold.
- **Task type:** Classification/extraction → small; reasoning → large.

6. Context Window Engineering

6.1 The Context Paradox

Larger context windows (200K+ tokens) enable comprehensive understanding but create a trap: developers fill available context indiscriminately, paying for irrelevant information that may actively *degrade* output quality. Research confirms LLMs are easily distracted by irrelevant context—more tokens does not equal better answers.

6.2 RAG Optimisation

Key optimisations for token-efficient Retrieval-Augmented Generation:

Contextual Chunking: Use an LLM to prepend brief contextual descriptions to each chunk before embedding, improving retrieval precision by 20–40%.

Hybrid Retrieval: Combine BM25 (exact lexical matching for names, IDs, acronyms) with semantic search, outperforming either method alone.

Autocut: Automatically limit results by detecting score gaps between relevant and irrelevant groups—replacing fixed top-K with a dynamic cutoff.

Chunk Expansion Post-Retrieval: Use small chunks (128–256 tokens) for precise retrieval and expand to surrounding context only after selection, decoupling search granularity from context provision.

6.3 Conversation Compaction

For multi-turn applications, conversation history grows linearly—creating unbounded cost growth. **Sliding Window with Summarisation:** Maintain full context for the most recent N turns; compress older turns. Claude Code uses this approach to enable effectively infinite conversations within fixed token budgets.

Extended Thinking: Thinking blocks from previous assistant turns are automatically excluded from input token counts in Claude. Use `budget_tokens` to cap reasoning expenditure per turn.

7. Speculative Decoding & Inference Optimisation

7.1 Speculative Decoding

Speculative decoding [11] generates draft tokens with a fast, small model and verifies them in parallel with the target large model. Since verification is parallelisable, this achieves **2–3× throughput improvement with zero quality loss**:

```
Small model: generates 5 draft tokens in 5 ms
Large model: verifies all 5 in parallel in 20 ms
             (vs. 100 ms sequential generation)
Result:      5 tokens in 25 ms = 4x speedup
```

The **Medusa** variant achieves 2.2× (Medusa-1) to 3.6× (Medusa-2) speedup [6].

7.2 Continuous Batching and vLLM

vLLM achieves **10–20× throughput** over naive dynamic batching via continuous batching at the iteration level and PagedAttention for KV-cache memory management [7]. Before optimisation, CPU consumed 62% of execution time versus only 38% GPU utilisation. vLLM v0.6.0 benchmarks: Llama 3 8B achieved 2.7× throughput improvement and 5× faster TPOT; Llama 3 70B achieved 1.8× throughput and 2× reduced TPOT.

7.3 Quantisation

Table 6: Quantisation Trade-offs for Self-Hosted Models

Precision	Memory	Quality Loss	Throughput
FP16 (baseline)	100%	0%	1×
INT8	50%	<1%	1.5–2×
INT4 (GPTQ/AWQ)	25%	2–5%	2–3×
GGUF Q4_K_M	25%	3–5%	2–3×

8. Fine-Tuning as a Cost Reduction Strategy

8.1 Eliminating Prompt Overhead

Fine-tuning converts per-request token overhead (system prompts, few-shot examples, format instructions) into model weights—amortising cost across all future requests.

Economic model:

$$C_{\text{base}} = (T_{\text{sys}} + T_{\text{ex}} + T_q) \cdot p \cdot N \tag{1}$$

$$C_{\text{ft}} = C_{\text{train}} + T_q \cdot p \cdot N \tag{2}$$

ROI is achieved when $C_{\text{train}} < (T_{\text{sys}} + T_{\text{ex}}) \cdot p \cdot N$.

Real example: 2,000-token few-shot examples at $3/MTok, 1M requests/month: without fine-tuning = $6,000/month in example tokens; fine-tuning cost ≈ $100 one-time. **Break-even: less than 1 day.**

8.2 Parameter-Efficient Fine-Tuning (PEFT)

Table 7: PEFT Method Comparison

Method	Params	Memory	vs. Full FT
LoRA [9]	0.1%	10–20%	Within 1–2%
QLoRA [10]	0.1%	48 GB (65B)	Within 2–3%
Adapters	3.6% add.	15%	Within 0.4%

8.3 Distillation

Distillation uses a frontier model to generate training data, then fine-tunes a model 10–100× cheaper. **Cursor:** custom distilled models at \$0.50/MTok vs. \$3/MTok—a 6× cost reduction with <5% quality degradation on code completions. **OpenAI model distillation:** GPT-5.5 outputs become training data for GPT-5.4-mini, enabling up to **25× input savings**.

8.4 The RAG + Fine-Tuning Anti-Pattern

OpenAI research revealed that combining RAG with fine-tuning *decreased* accuracy (87 → 83) compared to fine-tuning alone [8]. Rule: failures from *missing knowledge* ⇒ use RAG; failures from *inconsistent behaviour* ⇒ use fine-tuning; never both for the same task dimension.

9. Multi-Agent Architectures

9.1 Token Budget Implications

Simple query via Haiku: ∼500 tokens = \$0.001. Same query through a 3-agent Sonnet pipeline: ∼5,000 tokens = \$0.02. A **20× cost increase** for multi-agent vs. single-agent on simple tasks—justified only when quality improvement outweighs cost.

9.2 Sub-Agent Pattern

The most token-efficient multi-agent pattern: main agent maintains full context; sub-agents receive only task-relevant excerpts (80–90% context reduction per agent) and return compressed summaries of 1,000–2,000 tokens. Claude Code uses this: spawning focused sub-agents for git history, test context, and code search, each with minimal context.

9.3 Mixture of Agents

Together.ai's MoA research demonstrates a **7.6% absolute quality improvement** over GPT-4o (AlpacaEval 2.0: 65.1% vs. 57.5%) at the cost of higher token consumption. Six diverse proposers scored 61.3% vs. 56.7% with a single repeated model. MoA-Lite matches GPT-4o cost with superior quality.

10. Structured Output Optimisation

Requesting structured output (JSON, XML schemas) provides dual benefits: (1) eliminates format tokens—no preambles or markdown; (2) enables deterministic parsing. **Token savings: 30–60% fewer output tokens** vs. equivalent prose for factual/analytical tasks. Schema overhead is amortised across many calls.

Claude tool-use overhead: Bash tool adds +245 input tokens; text editor adds +700; computer use adds +735 per definition. Schema overhead is one-time; savings per call accumulate rapidly at scale.

11. Coding Assistant Optimisation

11.1 GitHub Copilot

Copilot processes **∼6,000 characters** per completion request using sophisticated context engineering [12].

Neighbouring Tabs: Processes all open IDE files using Jaccard similarity. Setting a very low bar for inclusion improved acceptance rate by **5%**.

Fill-in-the-Middle (FIM): Uses both prefix (code before cursor) and suffix (code after cursor) as context—a **10% relative performance boost** at no added latency through optimal caching.

11.2 Claude Code

Claude Code averages **\$6 per active developer per day** using:

- *Context compaction:* Automatic summarisation as conversation grows.
- *CLAUDE.md:* Project-level persistent memory without per-message cost.
- *Sub-agent architecture:* Focused agents with minimal context, returning 1,000–2,000 token summaries.
- *Context-awareness:* Explicit token budget: `35000/1000000; 965000 remaining` after each tool call.

11.3 Cursor

Cursor uses lightweight open-source models for classification/reranking and reserves proprietary API calls for actual completions. Custom distilled models at **\$0.50/MTok** vs. \$3/MTok for frontier—6× savings through distillation, with context engineering as their core differentiator.

12. Production Case Studies

RouteLLM deployment: 95% of GPT-4 quality achieved using only 26% GPT-4 calls—74% handled by Mixtral at 1/50th the cost.

Anthropic prompt caching: 79% latency reduction and 90% cost reduction on a 100K-token document analysis application.

AlphaCodium decomposition: Accuracy from 19% to 44% while individual steps used fewer tokens than the monolithic prompt.

Netflix/LiteLLM: A Netflix staff engineer reported LiteLLM "has saved us months of work" by eliminating manual model integration and enabling rapid deployment.

Cursor distillation: \$0.50/MTok vs. \$3/MTok—a 6× cost reduction with <5% quality degradation on code completions.

13. Quantitative Savings Reference

Table 8 provides a consolidated reference for expected savings per technique.

Table 8: Token Savings by Technique

Technique	Savings	Conditions
Prompt Caching	90% input	Repeated prefixes >1K
Semantic Caching	50–90% total	High query similarity
Batch API	50% all	Async workloads
Opus → Haiku	80% in/out	Within smaller model
Opus → Sonnet	40% in/out	Most production
Effort (high → low)	up to 90% think	Simple tasks
Distillation	96% input	Narrow tasks
Speculative decoding	2–3.6× TP	Self-hosted latency
Continuous batching	10–20× TP	vs. dyn. batching
Quantisation (INT8)	50% memory	Acceptable quality
Structured output	30–60% output	Extraction tasks
Sub-agent architecture	80–90% context	Complex agents
DeepSeek cache hit	98% input	Repeated prompts
FIM (coding)	10% quality gain	No extra cost

13.1 Compounding Savings Example

Starting with Claude Opus at \$5/\$25/MTok: (1) apply prompt caching (90% input savings) → effective \$0.50/\$25; (2) use Batch API (50% discount) → effective \$0.25/\$12.50; (3) route 70% of traffic to Haiku (80% further savings). **Combined effective cost reduction: 85–95% vs. naïve baseline.**

14. Unified Optimisation Framework

14.1 The Token Efficiency Stack

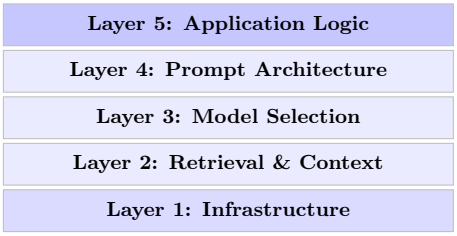

Figure 1: Five-layer Token Efficiency Stack. Each layer provides independent optimisation opportunities; compound savings can exceed 95%.

14.2 Implementation Roadmap

Phase 1 — Week 1 (immediate wins): Enable prompt caching (static content first); set per-task `max_tokens`; use Batch API for async workloads; set `effort`/reasoning budget controls.

Phase 2 — Month 1: Deploy model routing (simple classifier); optimise RAG (reranking, autocut, hybrid search); implement conversation compaction; add structured output schemas.

Phase 3 — Quarter 1: Fine-tune domain-specific models; build LLM cascades with confidence escalation; implement distillation pipelines; deploy speculative decoding for self-hosted models.

14.3 Decision Matrix

Table 9: Strategy Selection by Problem Type

Problem	Strategy	Savings	Effort
Repeated queries	Prompt+sem cache	80–90%	Low
High vol., simple	Model routing	70–85%	Med
Long system prompts	Fine-tuning	60–80%	High
Large context	RAG optimisation	40–70%	Med
Non-real-time	Batch API	50%	Low
Complex reasoning	Effort param.	60–90%	Low
Multi-turn chat	Compaction	50–75%	Med
Code gen. at scale	Distillation	80–95%	High

15. Future Directions

Adaptive inference: Models that dynamically allocate computation per-token based on difficulty—easy tokens get fewer transformer layers, hard tokens get full depth.

MoE scaling: Mixture-of-Experts architectures (e.g., Mixtral activates only 12B of 46B parameters per token) provide frontier quality at budget compute. This trend will continue and deepen.

On-device inference: Models running locally (phones, laptops) with zero per-token API cost for simple tasks, reserving cloud inference for complex reasoning.

Intelligent inference orchestration: We predict convergence toward systems that automatically select the optimal model, context size, reasoning depth, and caching strategy per request—analogous to how database query optimisers hide execution plan details from application developers.

16. Conclusion

Token optimisation is not about austerity—it is about **engineering precision**. Every token should earn its place in the context window. Every API call should be routed to the most cost-effective capable model. Every repeated computation should be cached.

The techniques documented herein—from prompt caching (90% savings) to model routing (85% savings) to fine-tuning (eliminating per-request overhead)—are production-proven and immediately implementable. A 90% cost reduction is not the ceiling; it is the starting point. Treat token budget as a first-class architectural constraint, apply optimisation at every layer of the stack, and compound savings multiplicatively.

References

[1] H. Jiang et al., "LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models," *EMNLP*, 2023.

[2] Z. Pan et al., "LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression," 2024.

[3] H. Jiang et al., "LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression," 2024.

[4] L. Chen et al., "FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance," *Stanford*, 2023.

[5] I. Ong et al., "RouteLLM: Learning to Route LLMs with Preference Data," *LMSYS*, 2024.

[6] T. Cai et al., "Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads," 2024.

[7] W. Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," *SOSP*, 2023.

[8] OpenAI, "Optimizing LLM Accuracy: Best Practices," 2024.

[9] E. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," *ICLR*, 2022.

[10] T. Dettmers et al., "QLoRA: Efficient Finetuning of Quantized Language Models," *NeurIPS*, 2023.

[11] Y. Leviathan et al., "Fast Inference from Transformers via Speculative Decoding," *ICML*, 2023.

[12] GitHub Engineering, "How GitHub Copilot Uses Context to Improve Code Suggestions," 2023.

[13] T. Ridnik et al., "Code Generation with AlphaCodium: From Prompt Engineering to Flow Engineering," 2024.

[14] S. Zitnik et al., "What We Learned from a Year of Building with LLMs," *O'Reilly*, 2024.

[15] Anthropic, "Prompt Caching Documentation," 2024–2026.